

Repenser l'agilité à l'âge des agents

David-Olivier Saban, avec l'aide de Claude Sonnet 4.6 et GPT-5.5

Repenser l'agilité à l'âge des agents

Version management condensée

Auteur : David-Olivier Saban, avec l'aide de Claude Sonnet 4.6 et GPT-5.5

Public cible : Engineering Managers, Heads of Engineering, Product Leaders, responsables de transformation.

Objectif : expliquer comment démarrer une transformation agentique sans la réduire à un achat d'outils IA.

Versions disponibles :

Document	Titre	Usage
Version management	Repenser l'agilité à l'âge des agents	Version condensée pour management, Heads of Engineering et Product Leaders.
Version longue	La mort du code manuel : repenser le SDLC à l'âge des agents	Version détaillée pour Engineering Managers et Tech Leads.

Executive Summary

L'IA ne transforme pas seulement l'écriture du code. Elle transforme le système de delivery.

Ce document est la version management condensée. La version longue, **La mort du code manuel : repenser le SDLC à l'âge des agents**, contient les détails techniques et les exemples complets.

Le message pour le management est direct : acheter des licences ne suffit pas. Il faut construire un setup agentique¹ : documentation versionnée, instructions courtes, tools² déterministes, CI/CD³ dimensionnée, environnements de PR⁴, code owners, guardrails et règles de validation.

La valeur apparaît quand toute l'équipe utilise le même flux : PM, CSM, QA, designers, développeurs et agents⁵. Le porteur du besoin valide le fonctionnel ; le code owner valide le technique.

¹**Agent** — Configuration spécialisée combinant un rôle, des règles de comportement, des interdictions et une capacité à utiliser des outils. Un agent développeur ne se comporte pas comme un agent QA ou PM.

²**Tool** — Action déterministe automatisée : créer un worktree, lancer l'application locale, exécuter la validation. Ce qui doit toujours s'exécuter de la même façon ne doit pas dépendre d'un prompt.

³**CI/CD** — Intégration Continue / Déploiement Continu. Pipeline automatisée qui compile, teste et déploie les changements.

⁴**Pull Request (PR)** — Proposition de changement soumise à validation dans un repository Git. Chaque changement, qu'il vienne d'un humain ou d'un agent, passe par une PR avant d'être intégré.

⁵**Skill** — Connaissance réutilisable mobilisable par plusieurs agents : écrire une user story, auditer une PR, conduire une revue de sécurité, maintenir la documentation.

Promesse Et Résultats Observés Sur Mon Projet Test

La promesse n'est pas seulement de coder plus vite. La promesse est de réduire la friction du delivery de bout en bout.

Sur mon projet test, encore en prototypage, les signaux sont déjà concrets :

Dimension	Résultat observé
Delivery	Une équipe beaucoup plus réduite livre autant qu'une équipe précédente d'environ douze personnes.
Coordination	Moins de handoffs, moins de réunions, décisions plus rapides.
CI/CD	La capacité est passée d'environ 3 builds simultanés à environ 20 builds simultanés, sans hausse équivalente des coûts grâce à l'infrastructure préemptible.
Monorepo	CI/CD, infra, docs, CMS, UI et tests E2E peuvent vivre ensemble, évitant jusqu'à six repos/pipelines/structures agentiques.
Petits changements	Un changement CMS type color picker a été écrit, codé, validé et mergé en moins d'une heure, avec environ 30 minutes de temps humain cumulé.
Quality of life	PM/CSM peuvent porter des irritants faibles risques avec agent, si le code owner valide la PR.
Qualité de PR	Les PRs bien préparées et bien contextualisées sont passées d'environ une dizaine de commentaires de review à souvent 0-2 commentaires.

Ces chiffres ne sont pas une promesse universelle. Ils montrent ce qui devient possible quand le terrain technique et organisationnel est prêt.

Thèse Pour Managers

Le vrai sujet n'est pas l'IA. Le vrai sujet est l'organisation du travail autour de l'IA.

Les gains apparaissent quand trois conditions sont réunies :

1. **Le context engineering⁶ est en place** : documentation, spécifications, décisions, workflows et contraintes sont versionnés et accessibles dans le repository — aux agents comme aux humains.
2. **Le setup est partagé** : développeurs, PM, CSM, QA et designers utilisent les mêmes références, les mêmes tools, les mêmes guardrails et un flux commun de la user story à la production.
3. **La validation est claire** : le porteur du besoin valide fonctionnellement ; le code owner valide techniquement.

Sans ces trois conditions, l'IA ajoute du bruit. Avec elles, elle augmente la vitesse, l'autonomie et la qualité du flux.

⁶**Context engineering** — Conception et maintenance de l'information nécessaire pour qu'un humain ou un agent travaille correctement : documentation, spécifications, décisions, contraintes, workflows.

Ce Qui Change Dans L'agilité

Les agents changent le rythme. Certaines tâches passent de plusieurs jours à moins d'une heure ; les décisions produit, architecture et priorisation restent humaines. Ce décalage crée une tension : les cérémonies conçues pour synchroniser un travail lent deviennent trop lourdes pour les petits changements, mais restent utiles pour arbitrer, prioriser et partager le risque.

Dans mon projet test, les cérémonies n'ont donc pas disparu. Elles sont devenues moins centrées sur le reporting et plus centrées sur la régulation du flux. Le standup sert à décider si un bug ou un irritant doit être pris par un développeur, un CSM avec Copilot, un agent autonome, ou reporté. Le refinement sert moins à écrire la solution qu'à clarifier l'impact, le risque et les critères d'acceptation. La retro sert à comprendre pourquoi un agent a échoué : contexte manquant, tool absent, test instable, instruction ambiguë.

Le tableau suivant résume cette adaptation :

Rituel	Décision management
Standup	Devenir un point de régulation : qui prend quoi, humain ou agent, avec quel risque.
Refinement	Se concentrer sur impact, risque, dette et réversibilité.
Sprint	Garder ce qui aide, mais absorber les petits changements en flux continu type Scrumban ⁷ .
Demo	Montrer plus souvent, idéalement sur environnement de PR.
Retro	Auditer les échecs agents : contexte manquant, tool absent, test instable, instruction ambiguë.

⁷**Scrumban** — Mode hybride : structure Scrum (standup, retro, vision produit) combinée à un flux Kanban continu pour les petits changements.

Le Setup Agentique Comme Source De Confiance

La confiance ne vient pas du fait que “l’IA est bonne”. Elle vient du fait que le système limite les erreurs et rend les responsabilités visibles.

Un setup agentique robuste contient au minimum :

Élément	Rôle
Documentation versionnée	Donne aux agents et aux humains le même contexte.
Instructions IA versionnées	Évitent les prompts personnels et contradictoires.
Agents par rôle	Développeur, PM, Product Design, QA, review, security.
Skills réutilisables	Story writing, review, testing, documentation, audit.
Tools déterministes	Worktree, validation, lancement local, génération rapports.
CI/CD dimensionnée	Absorbe le flux d’implémentation et de validation humain + agents.
Environnements de PR	Permettent validation fonctionnelle avant merge.
Code owners	Maintiennent l’accountability technique.
Standup de régulation	Rend visible qui prend quoi et avec quel mode de delivery.
Outillage commun	Donne à toute l’équipe accès au code, à GitHub, aux agents et aux mêmes références.
Formation non-dev	Permet aux PM, PO, QA, CSM et designers de tester, lire une PR et valider sur environnement de PR.

Deux exemples montrent pourquoi ce setup n’est pas cosmétique.

Premier exemple : GMS Runner. Sur ce projet open-source interne, un agent utilisé sans configuration agentique a produit un correctif puis l’a committé et poussé sans validation. Répéter les règles dans la conversation ne suffisait pas : l’agent les oubliait à l’itération suivante. Après ajout d’un **AGENTS.md** court et versionné, décrivant TDD/ATDD et l’interdiction de commit sans confirmation explicite, le comportement a changé : l’agent a écrit un test reproduisant le bug, corrigé, validé, constaté que le correctif était incomplet, puis itéré. Même modèle, même tâche, résultat radicalement différent.

Deuxième exemple : l’agent QA. Les agents développeurs livraient souvent du code correct, mais omettaient certains éléments des spécifications. Nous avons donc ajouté un agent QA indépendant, fonctionnel plutôt que technique. Il ne relit pas le code : il compare la user story à la PR, teste l’environnement de PR avec Playwright, puis produit un tableau des acceptance criteria passés ou échoués, avec preuves.

Le tableau suivant synthétise les principes à retenir :

Sujet	Décision
Instructions	Court, versionné, auditable. 600 lignes = environ 30 pages : trop pour un humain, trop pour un agent.
Exemple GMS Runner	Sans <code>AGENTS.md</code> , l'agent commit/push sans validation. Avec TDD/ATDD et règles de commit, même modèle, comportement radicalement différent.
Agent QA	Agent fonctionnel qui compare story et PR, teste l'environnement de PR, produit AC passés/échoués + preuves.
Non-développeurs	PM/CSM/QA peuvent contribuer, mais jamais hors flux : PR visible, validation fonctionnelle par le porteur du besoin, validation technique par code owner.
Outillage commun	L'outil peut changer ; l'accès au repository, aux agents, aux PR envs et aux critères de validation doit rester commun.
Formation	PM/PO/QA/CSM doivent savoir lire une PR, tester un PR env et commenter les acceptance criteria.

Cette séparation est le cœur du modèle : démocratiser l'exécution sans diluer la responsabilité.

Organisation Cible

Une organisation agentique efficace n'est pas forcément plus grande. Elle est plus explicite.

Le vrai gain n'est pas la réduction d'effectif. C'est la baisse du coût de coordination : moins de handoffs, moins de files d'attente internes, moins de synchronisation implicite. L'équipe prend plus vite les décisions et voit plus tôt les problèmes.

Cela ne veut pas dire que les compétences disparaissent. Le CSM, par exemple, n'existait pas dans l'équipe initiale et a été ajouté parce que la voix client devait être plus proche du delivery. À l'inverse, la QA est surtout transverse dans ce setup parce que le contexte est encore du prototypage. Dans un produit mature, critique ou fortement exposé, ce choix serait probablement insuffisant.

Sur mon projet test, la composition d'équipe a évolué ainsi :

Rôle	Avant	Après
Développeurs	3 FE + 3 BE	2.5 (full-stack)
QA	1 (100%)	0.2 (1j/semaine, expert transverse)
DevOps	1 (embarqué)	0.3 (support externe)
PM + PO	2 personnes distinctes	1 (rôles fusionnés)
UX Designer (UXD)	1	0.5
CSM	— (n'existait pas)	1 (rôle ajouté)
Total ETP (Équivalent Temps Plein)	~12	~4.5

Ce n'est pas une recette RH. C'est un contexte de prototypage où l'équipe a été recomposée autour du delivery. Le tableau suivant résume les implications organisationnelles :

Point	Implication
2.5 ETP dev incluent un junior	Documentation explicite + guardrails + PR review rendent l'onboarding plus sûr.
Moins de coordination	Moins de handoffs, décisions plus rapides, problèmes plus visibles.
Petite équipe moins résiliente	Prévoir backup, support transverse et accès rapide aux experts.
QA réduite dans ce contexte	Acceptable en prototype ; pas une recommandation pour un produit mature ou critique.

La bonne question n'est donc pas : “avons-nous encore besoin de tel rôle ?”. La bonne question est : “de quelle compétence avons-nous besoin, à quelle fréquence, et avec quel niveau de risque ?”

Besoin	Décision organisationnelle
Besoin 80-100% du temps	Intégrer la compétence dans l'équipe.
Besoin ponctuel mais critique	Avoir un expert externe accessible rapidement.
Besoin faible et récurrent	Former quelqu'un dans l'équipe, ou établir un accord clair avec une équipe transverse capable d'absorber les requêtes et de s'organiser.
Besoin inconnu	Expérimenter, mesurer, puis décider.

Les rôles deviennent partiellement des skills, mais ils ne disparaissent pas. QA, UX, DevOps, sécurité, legal ou architecture doivent être présents selon la fréquence et la criticité du besoin. L'agent QA aide à valider plus tôt ; il ne remplace pas une vraie responsabilité qualité.

Roadmap Concrète De Démarrage

Coût total de l'expérimentation : 6 à 8 ETP à plein temps pendant toute la durée (environ 10 à 16 semaines selon les phases).

Requirement 0 : L'Organisation Est-Elle Prête ?

Microsoft appelle cela le “Requirement 0”. C’est le prérequis évalué par l’équipe de support externe avant de décider si oui ou non ils investissent 3-4 ETP pour accompagner le changement.

L’expérimentation agentique exige :

- **Un trust à 200% du management.** Pas un accord tiède. Un mandat clair, assumé, qui survit au premier échec.
- **Des acteurs owners de leur sujet à 200%.** Des gens qui portent l’initiative avec conviction, pas des exécutants assignés.
- **Une motivation à toute épreuve.** L’expérimentation va mal se passer par moments. L’équipe doit vouloir continuer.
- **L’ownership de bout en bout.** L’équipe doit pouvoir expérimenter aussi loin que possible, résoudre ses problèmes elle-même ou accéder facilement à des experts pour se débloquer.
- **Pas de goulots d’étranglement organisationnels.** S’il faut passer par un process, demander des autorisations, attendre des semaines de validation et de priorisation pour chaque décision technique — on peut arrêter tout de suite. L’organisation n’est pas prête.

Si ces conditions ne sont pas réunies, l’investissement ne produira pas de résultat. Le Requirement 0 n’est pas une formalité : c’est un filtre honnête qui évite de gaspiller du temps et de la confiance.

Échouer au setup n’est pas un échec. C’est un apprentissage. Cela ne signifie pas qu’il faut arrêter le changement, mais identifier les blockers, comprendre comment débloquer la situation et avancer.

Les deux sponsors jouent des rôles complémentaires :

- **Le sponsor externe (Microsoft/AWS)** challenge l’équipe, accepte l’échec comme normal, et peut recommander de repousser la suite du projet si le terrain n’est pas prêt. Son rôle est de guider l’apprentissage, pas de forcer le résultat.
- **Le sponsor interne (management Amadeus)** escalade les problèmes organisationnels et débloque les process internes qui freinent l’équipe. Son rôle est de lever les obstacles que l’équipe ne peut pas résoudre seule.

Phase 0 : Alignement Management

Durée recommandée : 1 à 2 semaines.

Avant de lancer un pilot, clarifier l’objectif dominant :

Objectif dominant	Arbitrage associé
Apprendre	Accepter l'échec et la répétition.
Construire des agents autonomes	Ralentir temporairement la delivery pour investir dans le setup.
Transformer la méthode	Toucher aux rituels, rôles et responsabilités.
Prouver un Return on Investment (ROI)	Définir les métriques avant le pilot.
Livrer plus vite	Objectif de phase avancée, pas objectif initial.

La tentation management est de lancer le pilot avec une promesse de productivité immédiate. C'est le mauvais point d'entrée. Si l'équipe doit prouver le ROI avant d'avoir compris les échecs, stabilisé les agents et adapté son flux, elle optimisera la démonstration plutôt que l'apprentissage.

Décision : ne pas démarrer par "livrer plus vite". Le premier objectif est d'apprendre, stabiliser le setup et aligner les arbitrages. La vitesse vient ensuite, comme conséquence d'un système qui fonctionne.

Phase 1 : Préparer Le Terrain, Pas Tout L'Agentique

Durée recommandée : 2 à 4 semaines selon maturité existante.

Investissement humain : ~4 ETP (1 tech lead/architect, 1 DevOps, 1 PM/PDA pour audit et préparation repo, 1 expert agentic pour accompagner l'adoption).

Coût licences IA : variable selon outil (Copilot, Kiro, Claude Code, etc.) — compter environ 20-50€/utilisateur/mois selon l'outil choisi.

Il ne faut pas non plus tomber dans l'excès inverse : construire tout l'agentique avant d'expérimenter. On ne sait pas encore quels agents, skills ou tools seront réellement utiles. En revanche, on sait déjà que certains prérequis non-IA sont indispensables. Un agent échoue sur un environnement local cassé pour les mêmes raisons qu'un newcomer. Une CI/CD lente ou instable bloque les agents comme elle bloque les humains. Une documentation dispersée rend le contexte fragile.

Décision : ne pas construire tout l'agentique avant d'essayer. Préparer d'abord le terrain non-IA : environnement local, CI/CD, PR envs, documentation, tests, accès.

Préparer le terrain signifie :

Élément	Pourquoi c'est indispensable
Environnement local reproductible	Les développeurs et les agents doivent pouvoir tester leurs changements dans les mêmes conditions. Si l'environnement local ne fonctionne pas, l'agent échouera pour les mêmes raisons qu'un newcomer.

Élément	Pourquoi c'est indispensable
CI/CD fonctionnelle et stable	On ne construit pas un système qui accélère sans stabilité. La pipeline doit aussi pouvoir scaler pour absorber les pics de charge générés par les humains et les agents.
Classification des changements	La CI/CD doit distinguer un merge qui affecte l'application d'un merge qui ne l'affecte pas. Un changement de documentation peut publier un wiki ; un changement de dépendances ou de code peut générer une nouvelle version applicative.
Environnements de PR	Ils sont une clé de voûte de la confiance. Ils permettent aux PM, PO, QA, CSM et développeurs de tester avant merge, donc de valider plus tôt et de réduire les cycles de correction.
Documentation centralisée	Faire l'inventaire de la documentation produit est essentiel pour le futur onboarding des ingénieurs et des agents. L'information cachée tue la qualité.
Repositories connexes rapprochés	Si l'expérimentation exige de travailler sur plusieurs repositories, rapprochez-les ou mettez-les dans le même repository pour fluidifier les changements. Sinon, chaque repo ajoute coordination, divergence et friction.
Tests E2E stables	Il n'en faut pas beaucoup au début. Ils peuvent venir progressivement. Mais quelques tests fiables servent d'exemple, de base de validation et de preuve que l'expérimentation peut être contrôlée.
Accès aux outils	Cela va sans dire, mais c'est souvent oublié : si un humain peut faire des choses que l'IA ne peut pas faire faute d'accès, l'expérience ne fonctionnera pas. GitHub, Figma, documentation et métriques doivent être accessibles au système agentique.
Outils IA alignés	Les outils peuvent évoluer, de VS Code à Kiro ou autre, mais l'équipe doit partager les mêmes références, agents, instructions et accès. Sinon chaque rôle développe son propre shadow workflow.

Le périmètre doit être petit, complet et maîtrisé : frontend, backend, données, métriques, infrastructure, ownership clair. Le setup agentique doit émerger par adhésion et par preuve, pas par imposition initiale.

Phase 2 : Onboarder L'Agent

Durée recommandée : 2 à 4 semaines, puis amélioration continue.

Un agent doit être onboardé comme un nouveau membre d'équipe. Il a besoin de règles, d'exemples, d'accès, de limites et de feedback. La différence est qu'une correction apportée à son contexte peut ensuite bénéficier à toute l'équipe : humains, agents actuels et agents futurs.

Décision : onboarding l'agent comme un newcomer. L'objectif n'est pas encore de livrer plus vite ; c'est de comprendre ce dont l'agent a besoin pour travailler correctement.

L'onboarding agentique consiste à construire progressivement :

Brique	Contenu initial
Instructions projet	Règles minimales, versionnées et spécifiques au contexte local.
Agents de base	Développeur, PM/story writer, review, QA.
Tools déterministes	Création worktree, validation, lancement local.
Accès aux informations	GitHub, Figma, documentation, métriques.
Guardrails de sécurité	Ce que l'agent peut faire seul, ce qu'il doit demander, ce qu'il ne fait jamais.
Définition du done agentique	Code, tests, validation, documentation, PR claire.
Boucles de feedback	Tester, échouer, corriger, recommencer.
Bases de connaissances et exemples	Références ai-artifacts ⁸ , agents, skills et workflows dans lesquels l'équipe peut piocher et adapter.

Former aussi les non-développeurs : lire une PR, tester un PR env, vérifier les acceptance criteria, commenter clairement. Sans cela, le setup reste un outil de développeurs.

La phase doit rester organique. Les bases **ai-artifacts** servent d'exemples dans lesquels piocher, pas de framework imposé.

Phase 3 : Pilot Avec 1 À 2 Équipes

Durée recommandée : 6 à 8 semaines.

Composition recommandée par équipe pilot : 2 développeurs, 1-2 fonctionnels (QA/PDA/PM), quelques fonctions de support observatrices.

Choisir une équipe restreinte d'early adopters qui vont itérer rapidement pour apprendre. L'équipe doit inclure un tech lead impliqué, un PM/PO disponible, un QA ou validateur fonctionnel. Les fonctions de support (architecture, sécurité, DevOps) doivent suivre le pilot pour ensuite évangéliser le reste de l'organisation.

Support externe : Microsoft et AWS proposent d'embarquer 2 à 4 ingénieurs pendant les premières semaines d'adoption. Ce coût d'accompagnement fait partie du pricing d'adoption chez les fournisseurs cloud : ils savent que c'est le prix de la réussite.

Le pilot doit être assez réel pour produire des apprentissages utiles, mais assez limité pour éviter de mettre en risque l'organisation.

⁸**ai-artifact** — Framework open-source qui versionne, compose et audite agents, skills, tools et overlays d'un projet. Permet de réutiliser des bases upstream sans lock-in.

Commencer par trois types de travaux :

1. Petits bugs localisés.
2. Quality of life changes internes.
3. Documentation, tests et refactorings faibles risques.

Éviter au début : architecture structurante, sécurité critique, migration de données, auth, performance complexe, changements irréversibles.

Pour un produit critique, intégrer le focus QA dès le pilot : régression, critères d'acceptation, tests exploratoires, parcours critiques, qualité des releases.

Phase 4 : Adaptation Des Rituels

Durée recommandée : pendant le pilot.

Adapter sans tout renommer :

Rituel	Adaptation
Standup	Devient point de régulation du flux et de décision de délégation.
Refinement	Plus court, plus centré sur impact, risque, réversibilité.
Demo	Plus fréquente, souvent directement sur environnement de PR référencé dans la Pull Request.
Planning	Moins lourd pour les petits changements, priorisation continue.
Retrospective	Analyse des échecs agents, frictions setup, instructions manquantes.

Phase 5 : Mesure Et Décision De Scale

Durée recommandée : fin du pilot, puis mensuel.

Mesurer seulement ce qui guide la décision de scale :

Dimension	Métrique
Flux	Stories créées, PRs ouvertes, PRs mergées, validations terminées, releases.
Cycle time	Bug ouvert -> fix valide -> merge/prod.
CI/CD	Builds simultanés, temps d'attente, flaky rate.
Validation	Temps story -> PR -> environnement -> validation fonctionnelle -> merge.
Adoption	Usage agents/skills/tools par rôle.
Satisfaction	Feedback dev, PM, QA, CSM.
QA	Acceptance criteria validés/invalidés par rapport à la PR, régression, bugs trouvés avant/après merge.
Qualité	Incidents, rollbacks, commentaires critiques, tests cassants.

Passer à l'échelle seulement si le setup est stable, les objectifs clairs et les métriques lisibles. Sinon, continuer à apprendre.

Investissement Et ROI

L'expérimentation complète représente environ 3 mois d'investissement concentré :

- Phase 1 (1 mois) : ~4 ETP (tech lead/architect, DevOps, PM/PDA, support).
- Phases 2-3 (2 mois) : ~7 ETP (équipe pilot complète + support externe).
- **Total** : ~18 ETP-mois (équivalent de 18 personnes mobilisées 1 mois chacune).

Ce n'est pas un coût supplémentaire net : l'essentiel de cet investissement est du temps d'équipe existante redirigé vers l'expérimentation.

Le retour se matérialise par la réduction du coût de coordination et la capacité de l'équipe à livrer avec moins de personnes. Sur une équipe initiale de 10-12 personnes, le ROI se situe entre 3 et 6 mois après la fin de l'expérimentation. L'équipe réduite peut ensuite accélérer sur le projet en cours et le modèle peut être répliqué sur d'autres projets.

Risques Et Mitigations

Risque	Mitigation
Échec au setup initial	L'équipe n'arrive pas à structurer, récupérer l'information, ou combler les gaps techniques. Si le projet cible est trop gros ou trop complexe, ne pas forcer : choisir un projet plus petit, ou investir significativement plus de temps sur le setup initial. Mieux vaut repousser l'implémentation sur un gros projet que de l'aborder prématurément. Microsoft a documenté un success story sur le framework .NET avec des centaines de contributeurs ; leur approche mérite lecture (voir Références).
Résistance au changement	Ne pas chercher à convaincre tout le monde d'emblée. Commencer par des early adopters motivés et convaincus. Le scepticisme se radoucit par la multiplication de petits projets réussis, pas par des arguments théoriques.
Perte de compétence	Le risque existe. Mais la code review fait monter les compétences de programmation : c'est comme lire un livre et poser des questions dessus. Il n'y a pas de perte sèche de connaissance si les équipes travaillent sérieusement. En revanche, certains automatismes seront oubliés — comme lorsqu'on crée un script pour se simplifier la vie et qu'on oublie ce qu'il y avait dedans. C'est aux ingénieurs de garder un minimum d'hygiène de développement.
Turnover et interruptions	Sur mon projet test, le sachant IA a quitté l'entreprise en cours d'expérimentation. Les nouveaux interlocuteurs avaient d'autres compétences et d'autres façons de voir ; on a appris différemment et c'était enrichissant. Vacances longues, hackathons, contraintes sur plusieurs semaines : tout cela impacte la delivery, mais l'apprentissage se fait et on progresse. Une expérimentation n'est pas un sprint linéaire — elle absorbe les perturbations si le setup est versionné et documenté.

Sécurité Et Conformité

La question de la sécurité revient systématiquement dans les discussions management. Elle est légitime mais ne doit pas devenir un prétexte pour ne pas avancer.

Les points à adresser :

Sujet	Approche
Données dans les LLM	Les outils majeurs (GitHub Copilot, Claude Code, Kiro) ne retiennent pas le code soumis pour l'entraînement en mode entreprise. Vérifier les conditions contractuelles.
Propriété intellectuelle	Le code généré par un agent est soumis aux mêmes règles que le code écrit par un humain : il passe par une PR, il est reviewé, il appartient à l'entreprise.
Périmètre d'accès agents	Définir explicitement ce que l'agent peut faire seul, ce qu'il doit demander, ce qu'il ne fait jamais. Ces guardrails sont versionnés dans les instructions projet.
Conformité réglementaire	Pas différent d'un outil de développement classique. L'agent ne déploie pas en production sans validation humaine. Les mêmes contrôles s'appliquent.

La sécurité est un sujet de configuration, pas un blocker fondamental. Les fournisseurs cloud ont des offres entreprise avec les garanties nécessaires.

Critères De Continuation

Ne pas juger l'expérimentation sur le résultat du premier mois. Sur mon projet test, si nous avions fait le bilan à un mois, nous aurions tout arrêté. La migration à Azure avait été catastrophique, les agents ne fonctionnaient pas bien, tout allait de travers.

Le retour de notre management et de Microsoft a été : l'échec est normal, tout va bien. Maintenant qu'on a quelque chose qui fonctionne suffisamment, on expérimente et on apprend dessus, on fixe au fur et à mesure.

Et on y est arrivé. Dans la douleur, mais ça s'est fait — parce que l'équipe a été moteur et que le top management a fourni un support sans faille.

Les vrais critères de continuation ne sont pas des métriques de productivité au premier mois. Ce sont :

Critère	Signal positif	Signal d'alerte
Confiance dans le système	L'équipe utilise le setup naturellement, les agents produisent du travail utile.	L'équipe contourne le setup, retourne aux anciens flux.
Progrès mensuels visibles	Chaque mois apporte des améliorations concrètes : moins d'échecs, meilleure qualité, nouveau use case débloqué.	Stagnation sur plusieurs semaines sans apprentissage.

Critère	Signal positif	Signal d'alerte
Support management	Le management accepte l'échec comme apprentissage et maintient le mandat.	Le management demande un ROI immédiat et menace d'arrêter au premier problème.
Engagement équipe	Les early adopters sont enthousiastes et partagent leurs succès.	L'équipe subit l'expérimentation sans y croire.

Le go/no-go n'est pas binaire. C'est une évaluation mensuelle de la trajectoire, pas du résultat instantané.

Recommandations Pratiques

1. **Commencer par le setup, pas par les promesses.** Sans environnement fiable, l'agent échoue et l'équipe perd confiance.
 2. **Utiliser un monorepo quand c'est possible.** CI/CD, infrastructure, documentation, CMS, UI et tests E2E dans le même repo réduisent la synchronisation et donnent aux agents une vision d'ensemble.
 3. **Préférer trunk-based development, petites PRs et feature flags.** L'objectif est de réduire la durée de divergence.
 4. **Ne pas imposer un framework central.** La gouvernance doit guider, documenter et supporter. Les standards utiles s'imposent parce qu'ils prouvent leur valeur.
 5. **Donner un rôle aux non-développeurs dans le flux complet.** PM, CSM, QA et designers peuvent contribuer de la story à la validation si le setup est commun et si la validation technique reste claire.
 6. **Former les non-développeurs au flux GitHub.** Lire une PR, tester un environnement de PR, vérifier les acceptance criteria et commenter une validation doivent devenir des compétences d'équipe.
 7. **Traiter les échecs comme du diagnostic.** Un agent qui échoue révèle souvent un manque de contexte, d'environnement, de tests ou de guardrails.
 8. **Ne pas mesurer les lignes de code.** Mesurer le flux de bout en bout, la validation, la qualité et l'adoption.
-

Coût De L'Inaction

Ne pas expérimenter maintenant n'est pas une position neutre. C'est un choix avec des conséquences.

Le coût de l'inaction ne se mesure pas encore en euros — le ROI précis est en cours d'évaluation sur mon projet test, et le scale vers d'autres projets internes demande d'analyser comment chacun fonctionne pour adapter les méthodes. Ce travail viendra.

Ce qui se mesure déjà, c'est le retard d'apprentissage :

- **L'apprentissage est cumulatif et non-linéaire.** Les organisations qui expérimentent aujourd'hui accumulent des mois de savoir-faire sur le setup agentique, les échecs, les patterns qui marchent. Ce savoir ne s'achète pas — il se construit par l'itération.
- **L'écart se creuse chaque mois.** Chaque mois de pratique améliore les agents, les instructions, les tools et le flux. Plus on attend, plus il y aura de retard à rattraper.
- **L'attractivité se dégrade.** Les ingénieurs les plus compétents veulent travailler avec les outils modernes. Une organisation qui n'expérimente pas devient moins attractive.
- **La transformation subie est plus coûteuse.** Mieux vaut transformer à son rythme, en maîtrisant les choix, qu'être forcé de rattraper un retard sous pression.

L'expérimentation coûte ~18 ETP-mois. Ne pas expérimenter coûte un retard qui se mesure en trimestres, pas en semaines.

Message Final

L'IA ne rend pas une organisation performante par magie.

Elle amplifie ce qui existe déjà : contexte dispersé, CI/CD lente, review saturée, objectifs mal alignés.

Si le setup est solide, l'effet est puissant : développeurs plus sereins, non-développeurs dans le flux, QA plus tôt, irritants terrain traités, responsabilité technique conservée.

Le sujet n'est donc pas d'adopter un outil IA.

Le sujet est de construire un système de delivery où humains et agents peuvent travailler ensemble avec confiance.

Références

1. AWS DevOps & Developer Productivity Blog, **AI-Driven Development Life Cycle: Reimagining Software Engineering**, Raja SP, 31 July 2025. <https://aws.amazon.com/blogs/devops/ai-driven-development-life-cycle/>
2. Microsoft .NET Blog, **Ten Months with Copilot Coding Agent in dotnet/runtime**, Stephen Toub, March 2026. Success story documentant le déploiement à grande échelle sur un projet avec des centaines de contributeurs : 878 PRs, 67.9% merge rate. <https://devblogs.microsoft.com/dotnet/ten-months-with-cca-in-dotnet-runtime/>
3. Microsoft, **HVE Core**, repository utilisé comme source upstream pour des prompts RPI dans TSF. <https://github.com/microsoft/hve-core>
4. Monorepo Storefront utilisé comme projet test pour le framework **ai-artifacts**, qui versionne, audite et compose agents, skills, tools, overlays et bases de connaissances réutilisables. <https://github.com/amadeus-nexwave/discovery-travelstorefront-monorepo>